



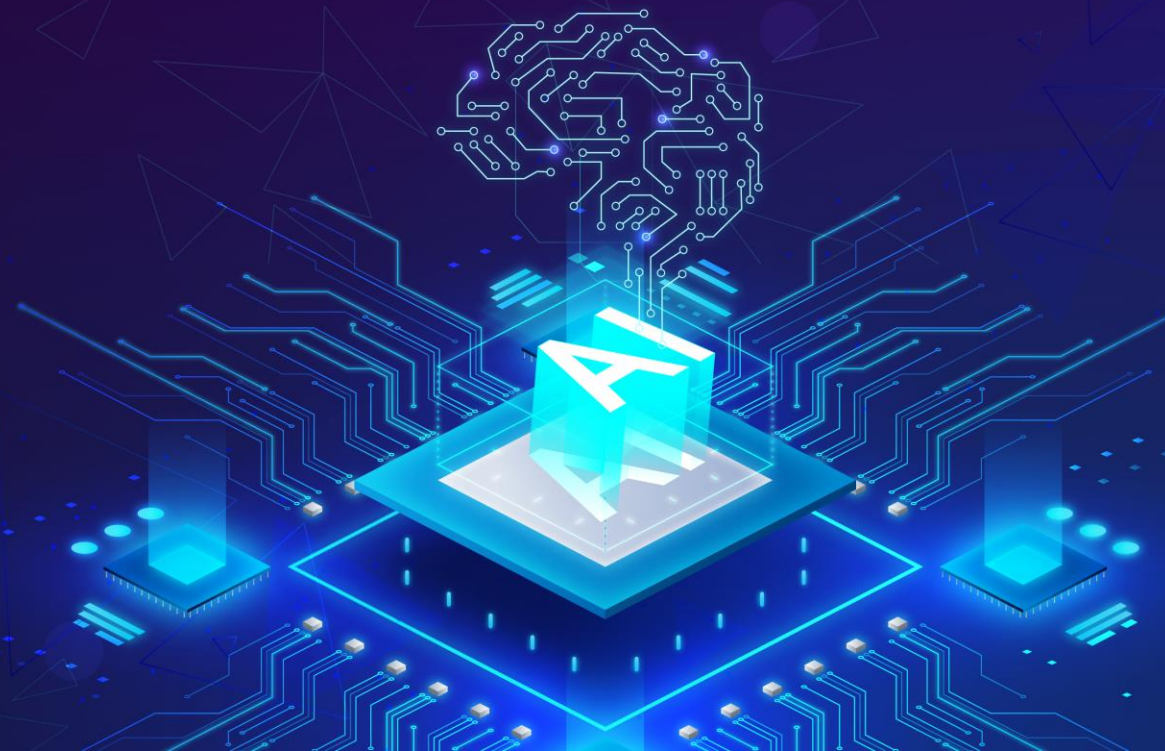
ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Nhóm chuyên môn Trí tuệ nhân tạo

NHẬP MÔN TRÍ TUỆ NHÂN TẠO

Hướng dẫn bài tập:

Tìm kiếm cơ bản



Bài tập tìm kiếm cơ bản

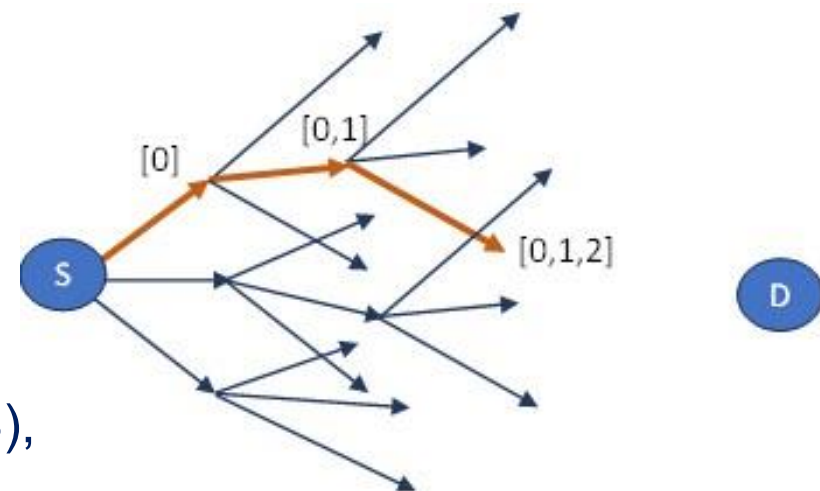
Bài học này nhằm giúp người học:

1. Hiểu rõ trình tự và đặc điểm của các giải thuật tìm kiếm cơ bản, bao gồm **BFS**, **UCS**, **DFS**, **LDS** và **IDS**.

Bài tập tìm kiếm cơ bản

▪ Bài tập 1. So sánh các phương pháp tìm kiếm cơ bản

- **Dữ liệu nhân tạo:** Các đường đi từ S. Một nút có 3 ngã rẽ ($b=3$).
S cách D (đích) không quá 15 nút ($m=15$).
- **Biểu diễn đường đi:** Chuỗi thứ tự các nhánh (nhánh 0, 1, 2)
Ví dụ [0, 1, 2] đi theo nhánh 0, sau đó 1, 2 như hình vẽ
- **Yêu cầu:** Tìm đường $S \rightarrow D$ với tìm kiếm cơ bản (BFS, DFS, IDS),
so sánh hiệu suất (thời gian và bộ nhớ) chạy chương trình.
- **Giả thiết:** Biết đường dẫn có đến được nút đích D hay không qua hàm kiểm tra.
Hàm kiểm tra (bạn không biết code, chỉ biết kết quả khi chạy hàm):



```
def check(path) -> bool:
    if path == [2, 1, 0, 2, 1, 0, 2, 1, 0, 2]:    #Đích D ở độ sâu 10
        return True
    return False
```

▪ Bài tập 1 (gợi ý)

- Hàm mở rộng: VD nút (đường dẫn) đầu vào: [0, 1, 2]
- Kết quả: đường dẫn theo 3 nhánh tiếp theo [[0, 1, 2, 0], [0, 1, 2, 1], [0, 1, 2, 1]]

```
def check(path) -> bool:
    if path == [2, 1, 0, 2, 1, 0, 2, 1, 0, 2]:    #Đích D ở độ sâu 10
        return True
    return False

def children(node):
    childNodes = []

    for branch in {0, 1, 2}:
        newNode = list(node)
        newNode.append(branch)
        childNodes.append(newNode)

    return childNodes
```

▪ Bài tập 1 (gợi ý)

- Hàm tìm kiếm theo chiều rộng (bfs)

- Xây dựng theo giả mã bài giảng

- NumNodes, MaxLen:
đếm số nút được mở rộng,
số nút lớn nhất của frontier

```
def bfs(root):  
    global MaxLen, NumNodes  
    frontier = [root]  
  
    while frontier:  
        MaxLen = max(MaxLen, len(frontier))  
        NumNodes += 1  
  
        n = frontier.pop(0)  
        if check(n):  
            return n  
  
        childNodes = children(n)  
        frontier.extend(childNodes)  
  
    return False
```

▪ Bài tập 1 (gợi ý)

- Hàm tìm kiếm giới hạn độ sâu (dls)
 - Xây dựng theo giả mã bài giảng
 - NumNodes, MaxLen:
đếm số nút được mở rộng,
số nút lớn nhất của frontier
 - depthLimit: độ sâu giới hạn

```
def dls(root, depthLimit = 15):  
    global MaxLen, NumNodes  
    frontier = [root]  
  
    while frontier:  
        MaxLen = max(MaxLen, len(frontier))  
        NumNodes += 1  
  
        n = frontier.pop()  
        if check(n):  
            return n  
  
        if len(n) < depthLimit:  
            childNodes = children(n)  
            frontier.extend(childNodes)  
  
    return False
```


▪ Bài tập 1 (gợi ý)

- Hàm tìm kiếm sâu dần (ids)
 - Xây dựng theo giả mã bài giảng
 - NumNodes, MaxLen:
đếm số nút được mở rộng,
số nút lớn nhất của frontier

```
def ids(root, maxDepth):  
    for depth in range(maxDepth):  
        node = dls(root, depth)  
        if node != False:  
            return node  
    return False  
  
MaxLen = 0  
NumNodes = 0  
result = ids([], 15)  
print("IDS algorithm")  
print("Path found:", result)  
print("Max frontier len:", MaxLen)  
print("Number of nodes explored:", NumNodes)
```

▪ Bài tập 1 (gợi ý)

- So sánh hiệu suất giải thuật
 - BFS, IDS: số nút khám phá nhỏ (dừng ở độ sâu lời giải $d = 10$)
 - DLS, DFS: số nút khám phá lớn (đến các nút độ sâu 15)
 - IDS, DLS: frontier nhỏ

BFS algorithm

Path found: [2, 1, 0, 2, 1, 0, 2, 1, 0, 2]

Max frontier len: 154.435

Number of nodes explored: 77.218

DLS algorithm

Path found: [2, 1, 0, 2, 1, 0, 2, 1, 0, 2]

Max frontier len: 31

Number of nodes explored: 4.138.904

IDS algorithm

Path found: [2, 1, 0, 2, 1, 0, 2, 1, 0, 2]

Max frontier len: 21

Number of nodes explored: 61.320

▪ Bài tập 2. Tìm kiếm cơ bản trên đồ thị

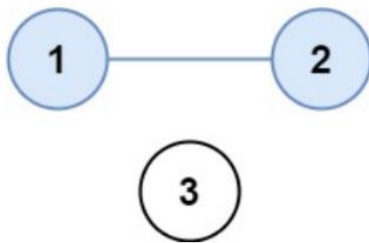
- <https://leetcode.com/problems/number-of-provinces/>
- There are n cities, some of them are connected, some are not.
- If a is connected with b , b is connected with c , then a is connected with c .
- A province is a group of connected cities and no other cities outside of the group.
- isConnected matrix ($n \times n$): $\text{isConnected}[i][j] = 1$ if city i and j are directly connected.
- Return the total number of provinces.

▪ Bài tập 2. Tìm kiếm cơ bản trên đồ thị

- Example 1:

- Input: isConnected = [[1,1,0],[1,1,0],[0,0,1]]

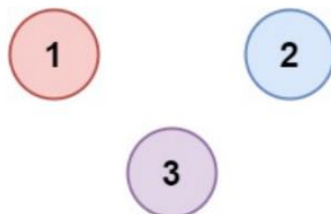
- Output: 2



- Example 2:

- Input: isConnected = [[1,0,0],[0,1,0],[0,0,1]]

- Output: 3



- Constraints:

- $1 \leq n \leq 200$

- $n == \text{isConnected.length}$

- $n == \text{isConnected}[i].\text{length}$

- $\text{isConnected}[i][j]$ is 1 or 0.

- $\text{isConnected}[i][i] == 1$

- $\text{isConnected}[i][j] == \text{isConnected}[j][i]$

▪ Bài tập 2 (gợi ý)

- Tìm kiếm trên đồ thị: Biến visited để tránh xét lặp các nút
- Hàm getNeighbors: mở rộng đến các city kết nối trực tiếp (~ children())

```
def search(self, root, isConnected) -> set:
    visited = set({})

    frontier = [root]
    while frontier:
        city = frontier.pop(0)
        visited.add(city)

        for neighbor in self.getNeighbors(city, isConnected):
            if neighbor not in visited:
                frontier.append(neighbor)
    return visited

def getNeighbors(self, city, isConnected: List[List[int]]) -> {set}:
    return {n for n in range(len(isConnected)) \
            if n != city and isConnected[city][n] == 1}
```

▪ Bài tập 2 (gợi ý)

- Gọi BFS/DFS để tìm tất cả các city nối với nhau (~ province = allConnectdCities)
- Biến countedCities để lưu các city đã thuộc về các province đã được đếm.

```
def findCircleNum(self, isConnected: List[List[int]]) -> int:
    count = 0
    countedCities = set()

    for city in range(len(isConnected)):
        if city not in countedCities:
            allConnectdCities = self.search(city, isConnected)
            countedCities.update(allConnectdCities)

            count = count + 1

    return count
```

1. Bài học đã cung cấp cho người học một số bài tập về tìm kiếm cơ bản nhằm minh họa **đặc điểm** và **ứng dụng** của các phương pháp tìm kiếm cơ bản.
2. Bài học đã trình bày **gợi ý**, **hướng dẫn** giải các bài tập tìm kiếm cơ bản đã cung cấp.

NHẬP MÔN TRÍ TUỆ NHÂN TẠO

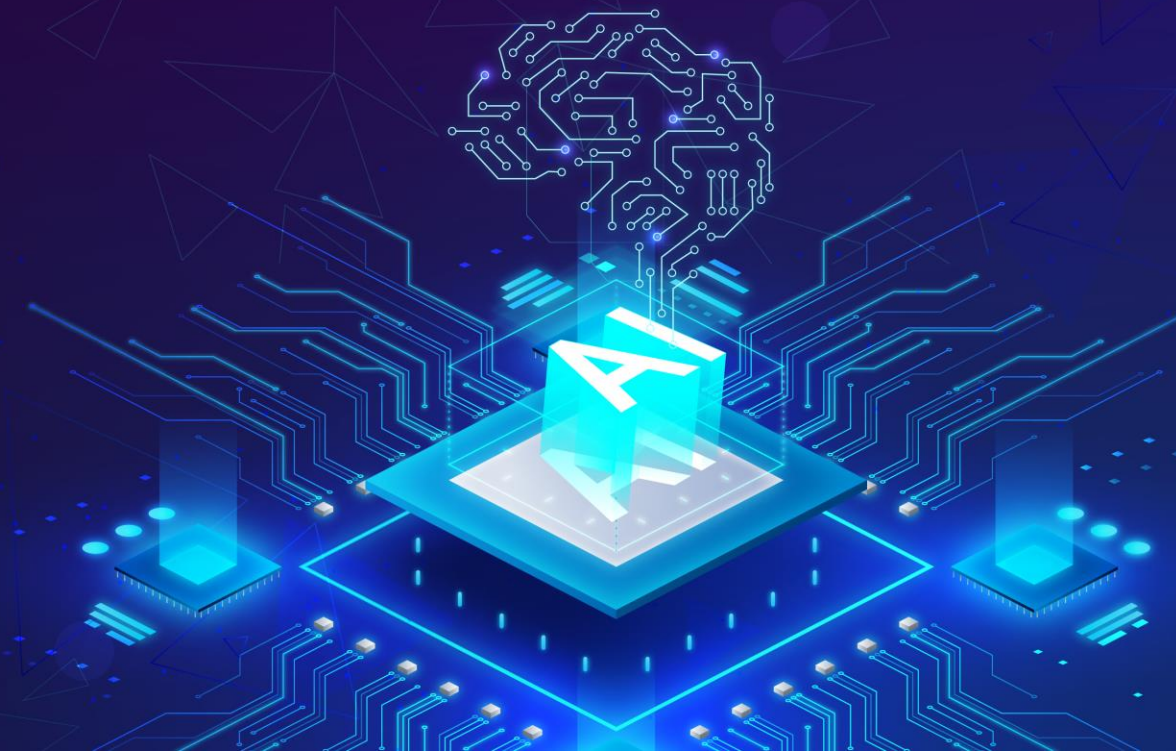
Hướng dẫn bài tập: Tìm kiếm cơ bản

Biên soạn:

TS. Đỗ Tiến Dũng

Trình bày:

TS. Đỗ Tiến Dũng



NHẬP MÔN TRÍ TUỆ NHÂN TẠO

Bài học tiếp theo:

Khái niệm về tìm kiếm có đối thủ

Tài liệu tham khảo:

- [1] S. Russell and P. Norvig. *Artificial Intelligence: A Modern Approach* (4th Edition). Prentice Hall, 2020
- [2] Từ Minh Phương. *Giáo trình nhập môn trí tuệ nhân tạo*. NXB Thông tin và truyền thông, 2016
- [3] T. M. Mitchell. *Machine Learning*. McGraw-Hill, 1997